



Mexus

Por tu seguridad,
Grabación de llamadas

dcueli.com

Documento de Arquitectura de
Software (SAD)

DAVID CUELI CHAVARRÍA

DOCUMENTO DE ARQUITECTURA DE SOFTWARE

Proyecto	Mexus
Tipo de documento	Corporativo – Documento de arquitectura de software (SAD)
Versión	1.1.0
Estado	Borrador inicial
Fecha	10/feb/2026
Rol	PO/Analyst
Autor	David Cueli

ÍNDICE

1. [Control de versiones](#)
2. [Cumplimiento normativo](#)
3. [Introducción](#)
 - 3.1. [Propósito del documento](#)
 - 3.2. [Alcance](#)
 - 3.3. [Arquitectura](#)
 - 3.4. [Documentos relacionados](#)
4. [Stakeholders y preocupaciones](#)
5. [Drivers arquitectónicos](#)
 - 5.1. [Drivers funcionales clave](#)
 - 5.2. [Drivers no funcionales clave](#)
6. [Visión arquitectónica](#)
 - 6.1. [Vista de contexto](#)
 - 6.2. [Vista lógica \(estructural\)](#)
 - 6.3. [Vista de componentes](#)
 - 6.4. [Vista de procesos \(dinámica\)](#)
 - 6.5. [Vista de datos](#)
 - 6.6. [Vista de despliegue](#)
 - 6.7. [Vista de seguridad](#)
7. [Descripción de componentes](#)
 - 7.1. [Capa de presentación \(Angular\)](#)
 - 7.2. [Capa de integración \(Capacitor\)](#)
 - 7.3. [Capa nativa Android](#)
8. [Gestión de datos](#)
 - 8.1. [Almacenamiento](#)
 - 8.2. [Metadatos](#)
9. [Gestión de errores y logging](#)
10. [Atributos de calidad y evaluación](#)
11. [Implicaciones y compromisos](#)
12. [Consideraciones de seguridad y privacidad](#)
13. [Riesgos arquitectónicos/técnicos conocidos](#)
14. [Trazabilidad](#)
15. [Conclusión](#)
16. [Aprobación](#)
17. [Estado del documento](#)

Información sobre **MEXUS**, de dcueli.com

- GitHub
<https://github.com/dcueli/Mexus>

CONTROL DE VERSIONES - ir al índice -

Versión	Fecha	Cambios realizados	Autor
<u>0.1.0</u>	10/02/2026	Documento inicial creado	dcueli
		• Cumplimiento normativo • Introducción • Stakeholders y preocupaciones • Drivers arquitectónicos • Visión arquitectónica	
	11/02/2026	• Descripción de componentes • Gestión de datos • Gestión de errores y logging • Atributos de calidad y evaluación • Implicaciones y compromisos • Consideraciones de seguridad y privacidad • Riesgos arquitectónicos/técnicos conocidos • Trazabilidad • Conclusión • Aprobación • Estado del documento	dcueli
<u>1.0.0</u>	11/02/2026	Revisión I de documento	dcueli
<u>1.1.0</u>	12/02/2026	Revisión II de documento	dcueli
	16/02/2026	Revisión III de documento	dcueli
	19/02/2026	DOCUMENTO APROBADO	dcueli

CUMPLIMIENTO NORMATIVO - ir al índice -

El documento se ha elaborado conforme a las siguientes normas y estándares:

- **IEEE 1016-2009** – Systems and Software Engineering – Software Design Description
- **ISO/IEC/IEEE 42010:2011** – Systems and software engineering — Architecture description
- Alineado conceptualmente con **ISO/IEC 25010** para atributos de calidad

INTRODUCCIÓN - ir al índice -

Propósito del documento

El presente documento describe la arquitectura de software del sistema **Mexus**, proporcionando una visión completa y técnica de alto nivel que permita comprender, implementar, mantener y auditar el sistema, justificar y defender las decisiones arquitectónicas adoptadas.

El SAD traduce los requisitos definidos en el SRS (IFR / INFR) a una solución arquitectónica coherente.

El documento está orientado a:

- Equipo de desarrollo.
- Equipo de pruebas.
- Responsables técnicos y de producto.

Y su objetivo es:

- Servir como referencia técnica autoritativa
- Permitir la evaluación de la arquitectura por un comité técnico
- Garantizar la trazabilidad entre requisitos, decisiones y soluciones
- Facilitar mantenimiento, evolución y auditoría

Alcance

El SAD cubre:

- Vistas arquitectónicas relevantes
- Decisiones arquitectónicas clave y alternativas evaluadas
- Escenarios arquitectónicos
- Implicaciones, compromisos y riesgos

Quedan fuera del alcance:

- Planificación del proyecto
- Estrategias de pruebas detalladas
- Gestión operativa

Arquitectura

La arquitectura definida en este documento cubre:

- La aplicación móvil Mexus (Android)
- La capa de presentación (Angular)
- La capa de integración nativa (Capacitor)
- La interacción con APIs nativas de Android para estado de llamadas y grabación de audio

Quedan explícitamente fuera:

- Backend, servicios remotos o sincronización
- Persistencia externa o en la nube
- Integraciones con terceros

Documentos relacionados

- SRS – Software Requirements Specification
- Plan de Pruebas / Estrategia de Testing
- Matriz de Trazabilidad
- Plan de Proyecto

STAKEHOLDERS Y PREOCUPACIONES - ir al índice -

<i>Stakeholder</i>	<i>Preocupaciones principales</i>
Arquitecto	Coherencia, calidad, evolución
Desarrollo	Claridad estructural, mantenibilidad
QA	Testabilidad, trazabilidad
Seguridad	Acceso a recursos, protección de datos
Dirección técnica	Riesgo, coste, sostenibilidad

DRIVERS ARQUITECTÓNICOS - ir al índice -

Derivados directamente del SRS:

Drivers funcionales clave

- Grabación manual y explícita de llamadas (*IFR_01*)
- Persistencia de grabación en segundo plano (*IFR_02*)
- Almacenamiento local de grabaciones (*IFR_05*)
- Asociación de metadatos básicos (*IFR_07*)
- Listado y reproducción de grabaciones (*IFR_08, IFR_09*)

Drivers no funcionales clave

- Estabilidad durante llamadas (*INFR_01*)
- Impacto mínimo en recursos (*INFR_02*)
- Privacidad y almacenamiento exclusivamente local (*INFR_04, INFR_05*)
- Arquitectura modular y desacoplada (*INFR_11, INFR_12*)
- Compatibilidad Android 10+ (*INFR_09*)

VISTA ARQUITECTÓNICA - ir al índice -

Vista de contexto

El sistema **Mexus** se ejecuta como una **aplicación de escritorio local**, interactuando con:

- Sistema Operativo (audio, permisos)
- Usuario final
- Sistema de archivos local

No depende de servicios externos en su núcleo funcional.

Vista lógica (estructural)

La arquitectura sigue un **modelo en capas estrictamente separadas**:

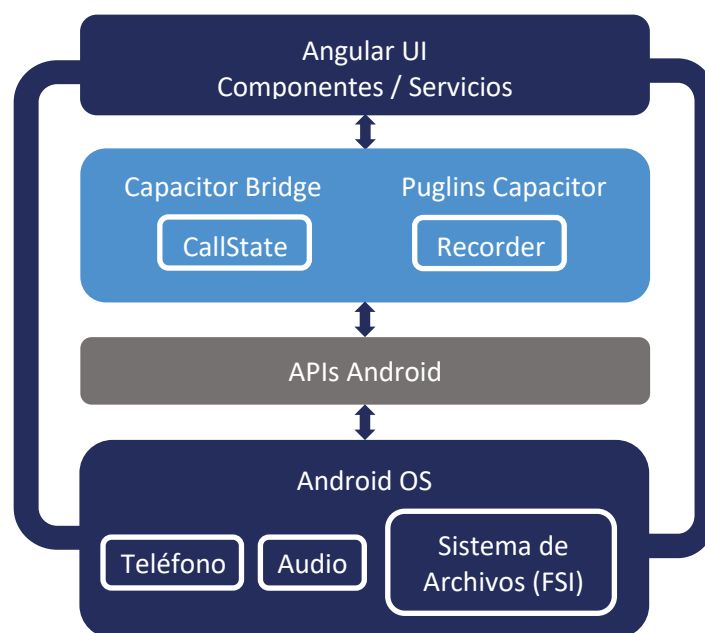
1. **Capa de presentación** (Angular)
2. **Capa de integración** (Capacitor Plugins)
3. **Capa nativa Android** (Java)

Justificación

- Aislamiento de la lógica de negocio.
- Independencia tecnológica.
- Facilita pruebas unitarias y de integración.

Alternativas evaluadas

- Arquitectura monolítica sin capas (*descartada por mantenimiento costoso*).
- Arquitectura cliente-servidor (*descartada por inicialmente innecesaria y más costosa*).
- Arquitectura hexagonal completa (*descartada por complejidad innecesaria*).



Vista de componentes

Componentes principales

- **UI Module:** interacción con el usuario
- **Application Controller:** orquestación de casos de uso
- **Call State Manager:** gestión de estados de llamada
- **Recorder Core:** control de grabación
- **Plugin Manager:** carga y aislamiento de extensiones
- **Audio Adapter:** abstracción del SO
- **Persistence Manager:** almacenamiento
- **Error & Logging Module:** gestión transversal de errores

Justificación

La descomposición permite asignar responsabilidades únicas y minimizar dependencias cruzadas.

Vista de procesos (dinámica)

El sistema opera bajo un **modelo orientado a eventos**:

- Eventos de usuario
- Eventos del sistema operativo
- Eventos internos de estado

La grabación de audio se gestiona en **procesos/hilos separados** para garantizar:

- No bloqueo de la UI
- Latencia controlada

Riesgo mitigado

Congelación de interfaz y pérdida de datos.

Vista de datos

Entidades principales

- CallSession.
- Recording.
- AudioStream.
- Metadata.

Los datos se almacenan localmente, separados entre:

- Datos binarios (audio).
- Metadatos estructurados.

Justificación

- Simplifica recuperación.
- Reduce corrupción de datos.

Vista de despliegue

- Aplicación de escritorio.
- Instalación local.
- Directorios separados para binarios, datos y plugins.

Alternativa descartada

Arquitectura cliente-servidor (*descartada por inicialmente innecesaria y más costosa*).

Vista de seguridad

- Acceso controlado a APIs del SO
- Permisos mínimos necesarios
- Aislamiento de plugins

Justificación

Reducir superficie de ataque y riesgos derivados de extensiones de terceros.

DESCRIPCIÓN DE COMPONENTES - ir al índice -

Capa de presentación (Angular)

Responsabilidades

- Interfaz de usuario.
- Gestión de estado de la aplicación.
- Orquestación de acciones del usuario.

Componentes clave

UI Module y **Application Controller** son los componentes clave que describen el conjunto de:

- Pantalla de llamada activa.
- Pantalla de grabaciones.
- Pantalla de configuración.

La capa Angular

- No accede directamente a APIs nativas.
- Consume exclusivamente servicios expuestos por plugins Capacitor.

Capa de integración (Capacitor)

Se definen **plugins nativos separados**, alineados con *IFR_04*:

Plugin CallState

Responsabilidades:

- Detectar cambios de estado de llamadas (entrante, saliente, activa, finalizada)
- Notificar eventos a la capa Angular

Plugin Recorder

Responsabilidades:

- Iniciar y detener grabaciones
- Gestionar ficheros de audio
- Exponer información básica de la grabación (ruta, duración aproximada, estado)

Capa nativa Android

Responsabilidades

- Interacción con APIs del sistema operativo
- Gestión de permisos
- Acceso al sistema de archivos
- Manejo de audio y estado de llamadas

Restricciones

- Uso exclusivo de APIs públicas soportadas
- Cumplimiento estricto de políticas de Android 10+

ESCENARIOS ARQUITECTÓNICOS - ir al índice -

Escenario 1: Inicio de grabación durante llamada activa

- UI envía evento
- Application Controller valida estado
- Recorder Core inicia captura
- Audio Adapter interactúa con SO

Atributos cubiertos

Rendimiento, fiabilidad

Escenario 2: Fallo en plugin externo

- Error capturado por Error Module
- Registro persistente
- Notificación controlada a UI

Atributos cubiertos

Robustez, mantenibilidad

DECISIONES ARQUITECTÓNICAS CLAVE (ADR) - ir al índice -

Arquitectura modular con plugins

- **Alternativa:** código integrado
- **Decisión:** plugins
- **Impacto:** mayor complejidad inicial, mayor extensibilidad

Procesamiento de audio asíncrono

- **Alternativa:** procesamiento síncrono
- **Decisión:** asíncrono
- **Impacto:** mejora UX, complejidad de concurrencia

Gestión centralizada de errores

- **Alternativa:** manejo local
- **Decisión:** centralizado
- **Impacto:** trazabilidad y consistencia

Decisión	Justificación
Angular + Capacitor	Reutilización de conocimientos y portabilidad
Plugins separados	Mantenibilidad y desacoplamiento
Almacenamiento local	Privacidad y cumplimiento legal
Arquitectura en capas	Claridad y extensibilidad

GESTIÓN DE DATOS - ir al índice -

Almacenamiento

- Almacenamiento exclusivamente local.
- Ruta controlada por la aplicación.
- Sin sincronización, ni backup externo, ni ninguna otra posibilidad de exportación, compartición y/o distribución, o divulgación automática o no.

Metadatos

Los metadatos se almacenan asociados a cada grabación:

- Fecha y hora
- Duración
- Tipo de llamada
- Número (si disponible)

GESTIÓN DE ERRORES Y LOGGING - ir al índice -

La arquitectura incorpora:

- Manejo explícito de errores en cada capa.
- Propagación controlada de errores hacia la UI.
- Logging mínimo en capa nativa y Angular.

El logging:

- Está orientado a diagnóstico.
- No impacta significativamente en rendimiento.
- No expone datos sensibles.

ATRIBUTOS DE CALIDAD Y EVALUACIÓN - ir al índice -

Atributo	Estrategia
Rendimiento	Procesos separados, buffers
Seguridad	Permisos mínimos, aislamiento
Mantenibilidad	Capas, interfaces
Testabilidad	Separación de responsabilidades

IMPLICACIONES Y COMPROMISOS - ir al índice -

- Incremento de complejidad estructural inicial.
- Necesidad de disciplina en definición de interfaces.
- Mayor esfuerzo de diseño frente a soluciones rápidas.

CONSIDERACIONES DE SEGURIDAD Y PRIVACIDAD

- ir al índice -

- No transmisión de datos
- Acceso restringido a grabaciones
- Acciones siempre explícitas del usuario
- Preparación para cifrado futuro (sin implementación en 0.1.0)

RIESGOS ARQUITECTÓNICOS/TÉCNICOS CONOCIDOS

- ir al índice -

- Limitaciones del SO para grabación de llamadas.
- Comportamientos específicos por fabricante.
- Gestión de permisos en segundo plano.

Riesgo	Mitigación
Dependencia del SO	Abstracción por adaptadores
Errores en plugins	Aislamiento y control
Concurrencia	Diseño orientado a eventos

Estos riesgos se detallarán y ampliarán en el **Registro de Riesgos**.

TRAZABILIDAD - ir al índice -

Las decisiones arquitectónicas están trazadas a:

- IFR_01 – IFR_13 del SRS
- INFR_01 – INFR_12 del SRS
- Escenarios arquitectónicos
- Estrategia de pruebas

La trazabilidad detallada se formalizará en la **Matriz de Trazabilidad**.

CONCLUSIÓN - ir al índice -

La arquitectura definida para **Mexus** es coherente, defendible y alineada con estándares IEEE/ISO. Proporciona una base sólida para el desarrollo, evaluación y evolución del sistema, equilibrando calidad, coste y riesgo.

APROBACIÓN - ir al índice -

Aprobado por:

- *Nombre:* **David Cueli Chavarría**
- *Rol:* **Patrocinador / Responsable de Proyecto** en dcueli.com
- *Fecha:* **11/feb/2026**

ESTADO DEL DOCUMENTO - ir al índice -

- [Versión: 1.1.0](#)
- Estado: **APROBADO**